The Agent as a Principal

What a Coding Agent Changes in the Chain from Commit to Production, and the Rule That Restores It

Vasili Gavrilov
Software Architect and Full-Stack Engineer

15 October 2026

Abstract

In October 2026 an open letter from the scientific community, with more than four hundred signatories, asked that AI models able to attack computer systems be kept off outside networks by default. This paper is about a smaller case: the coding agent already running on many developers' machines. A coding agent is a program that writes code and runs commands under its developer's account. Before agents, the chain from a commit to production had two gates held by people: a review of the commit, and a release act that needs a release permission. The agent weakens the chain at one point. It acts on text its developer did not write: in a 2024 benchmark it followed a planted instruction 24% of the time. On a machine whose owner can release, the agent holds the release permission too, so a planted instruction can reach production with no person's act. This paper proposes one rule. If a developer can release, the agent on that developer's machine runs under an account of its own, and that account cannot release; and the release act asks for something only a person can supply. With the rule in place the agent can do no more than a developer who may commit and may not release, a case every team was handling before agents. Three things here are new. The finding of where the chain breaks: only on the machines of releasers, and only when a program can pass the release act. The paper gives the test for that. The rule with its condition, which names the machines that need it. And the recipe that puts it in place, with a check for each clause. The tools are old: least privilege and separation of duty, from 1975 and 1987, applied until now to people and to pipelines and here to the agent. The rule leaves review as the weakest link, where it was before agents.

1 Before Agents: The Chain

A team has people and programs. A permission system assigns to each identity x a set of acts $\mathrm{perm}(x)$ that it carries out on x's request. A principal is an identity the permission system decides about [1]; this paper says identity. x holds an act when the act is in $\mathrm{perm}(x)$. One act matters here above the others: the release r, which puts a built artifact into production.

Every act has two parties. The identity is the one the permission system sees and checks against perm. The issuer is whoever composed the request: a person at a keyboard, or a program.

Definition 1 (Person's act)**.** An act is a person's act when its issuer is a person who holds the act's identity, or a program whose every instruction such a person wrote. An act that requires an input a person supplies at the moment of the act has that person as its issuer.

A deploy script the developer wrote and a scheduler runs issues a person's act. A command an agent composes is not a person's act, whether the instruction came from the developer's task or from a file the developer never read; it becomes one if a person supplies the input it needs at that moment.

Definition 2 (Chain)**.** A chain to production is a sequence of acts, by any identities, that ends with a change in production. The chain starts with x when its first act is x's.

Before agents every act is a person's act, and the chain from a commit to production has two gates:

1. **Review.** A person other than the author approves the commit.
2. **Release.** A person holding r issues the release act.

Saltzer and Schroeder stated least privilege in 1975: every program and every user holds only the rights its task needs, and they state it for programs as well as for people [1]. Clark and Wilson stated separation of duty in 1987: the author and the releaser are different persons [2]. The supply-chain literature names the author, the reviewer, the builder and the deployer as identities of their own [16, 17, 14]. Together the two gates give the chain one invariant.

Invariant 1. Every change in production is made by a release act whose identity holds r and whose issuer is a person.

The invariant does not say the change is good. A malicious committer whose code passes review reaches production through the releaser's act, and the chain is as strong as its review. Where review is absent, the invariant still holds: the releaser's act is the last gate, and a committer needs the releaser.

2 What Changed: The Agent

A coding agent A is a program run by a developer d in d's shell. It reads files, edits them and runs commands, in a loop, toward a goal written in plain words. Two facts about it are new.

Its permission is the developer's. The agent's commands carry d's credentials, so the permission system sees d and $\mathrm{perm}(A) = \mathrm{perm}(d)$. The harness that runs the agent filters commands against rules of three kinds, deny, ask and allow. d sets them, and they match on the spelling of the command. The vendor's documentation says such a rule covers the invocation the model usually produces and is not a security boundary around the program [12]. The rules are per command. The permission system is per identity, and it sees d.

Its instructions are partly a stranger's. The agent takes instructions from everything it reads: the task, the files, web pages, error messages, comments in code written by others. Greshake et al. showed in 2023 that an instruction planted in text the model reads is followed as if the user had given it [5]. In InjecAgent, a GPT-4 agent obeyed a planted instruction in 24% of its valid responses, and in 47% when the text urged compliance [6]. In AgentDojo, a second model that screens inputs brings the rate of a GPT-4o agent to 8%, from about half without it [7]. Newer models score lower on the same benchmarks; the argument of this paper needs only a rate above zero. Those benchmarks run assistants for email, banking and shopping. For a coding agent the case is on record: in August 2025 a code editor's agent was driven by planted text to write its own settings file and run a command with no approval, CVE-2025-54135, scored 9.8 of 10 [10]. CaMeL

rebuilds the agent so that text it reads cannot choose which commands run and cannot flow into their arguments, and solves 77% of the AgentDojo tasks against 84% undefended [8]; Progent checks each command against a written policy [9]. The harness vendors add a sandbox that denies files and hosts whatever the command's spelling, and an administrator can lock it [12]. Each defence lowers the rate or removes one class of attack by deciding which command runs, or which file and host it reaches. None of them changes the account the command runs under: the cloud still sees d.

The agent is therefore an identity with d's permission, and in some fraction of its steps its acts come from text no person on the team wrote. Hardy named this shape in 1988: the confused deputy, a program that holds a person's authority and uses it on instructions the person did not give [3]. This deputy sits in the developer's own shell and holds the developer's keys, and in the 2026 Stack Overflow survey 65.9% of developers use an AI coding assistant or agent at work [11]. The open letter of October 2026, with 483 signatories by 15 October, asks that models able to attack systems be air-gapped by default [15]. This paper covers the agent that is already on the developer's machine and on the network because the work needs it, which is the user's side of the letter's concern.

3 What Weakened the Chain

Three assumptions carry the claims of this paper.

A1. The permission system decides on the identity of a request and on nothing else; the spelling of the command, and any wrapper round it, do not enter.

A2. An input is program-suppliable on a machine when a program on that machine can read or compute it.

A3. An identity's shell can read only that identity's credentials.

Claim 1. Let d hold r and run an agent A. Suppose the release act can be issued by a command from d's shell, and every input it requires is program-suppliable on d's machine. Then there is a chain to production that starts with A and contains no person's act.

Argument. A composes the release command. The command carries d's credentials, so by A1 the permission system sees d, and $r \in \mathrm{perm}(d)$, so it carries the act out. Each input the act requires is supplied by A, by A2 and the hypothesis. The chain is that one act. Its issuer is A, and by Definition 1 it is no person's act. □

The hypothesis holds in common practice; Table 1 shows why. Three checks in common use ask for things a program supplies: a harness prompt asks for the developer's yes, and for nothing once the rule is widened or prompts are off; a confirmation typed back asks for a line of text; a cached sign-in token asks for a file. The sign-in that produced the token was a person's act, made before the chain began, and it is not in the chain. Three checks ask for things a program does not supply: a phone code at the moment of release, the touch of a hardware key, a second person. Where the release act has one of these, the hypothesis fails and the claim does not apply.

The first row fails in two ways. The agent can write a program and have an allowed command, the test runner or the build, run it; the harness then never sees the release command. And the ask rule itself does not last. A person issues a few commands an hour, and a prompt on each is bearable, so the allow list stays short. An agent issues commands at machine pace, and each prompt is answered

Table 1: Checks on the release act, what each asks for, and whether it is program-suppliable on the developer's machine.

Check	What it asks for	Program-suppliable
Approval prompt in the harness	The developer's yes, or nothing once the rule is widened or prompts are off	yes
Confirmation typed back by the release script	A line of text	yes
Cached sign-in token	A file, a socket or a keyring, valid for the credential's lifetime	yes, within that lifetime
Phone code at the moment of release, where the sign-in offers one	The phone	no
Signature with a key on a hardware device that must be touched	The touch	no
Approval by a second person	That person	no

by typing a broader pattern so that it does not come back. The harness also offers a flag that switches prompts off [12]. Akhawe and Felt measured the same habit in browsers in 2013: users clicked through 70% of one browser's certificate warnings [4].

The invariant fails exactly on the machines of releasers whose release act is in the first three rows. A change can reach production under d's credentials with no person's act behind it, and the log names d. On the machine of a developer without r the release act is unchanged: $\mathrm{perm}(A) = \mathrm{perm}(d)$ excludes r, and the agent's reach is that developer's reach. The review gate is unchanged everywhere: the agent can commit bad code, as any committer can, and that code still needs a person's release.

4 The Rule

Rule. For every developer d with $r \in \mathrm{perm}(d)$ who runs an agent A:

(a) A runs under an identity a of its own, personal to d, with $r \notin \mathrm{perm}(a)$, and no credential that holds r is readable from a's shell.

(b) An owner above d holds the denial of r to a, so d cannot widen it.

(c) The release act asks for an input only a person can supply, whatever identity issues it.

(d) Every change a release makes passes through that one act, and production reads from no store that a can write.

For a developer with $r \notin \mathrm{perm}(d)$, nothing is required.

Claim 2 (The agent cannot release)**.** Under (a) and (b), no act issued from a's shell is a release act.

Argument. By (a) and A3, every act issued from a's shell carries a's credentials, so its identity is a. $r \notin \mathrm{perm}(a)$, and (b) keeps this true against later widening. By A1 the act is refused however the command is written or wrapped. □

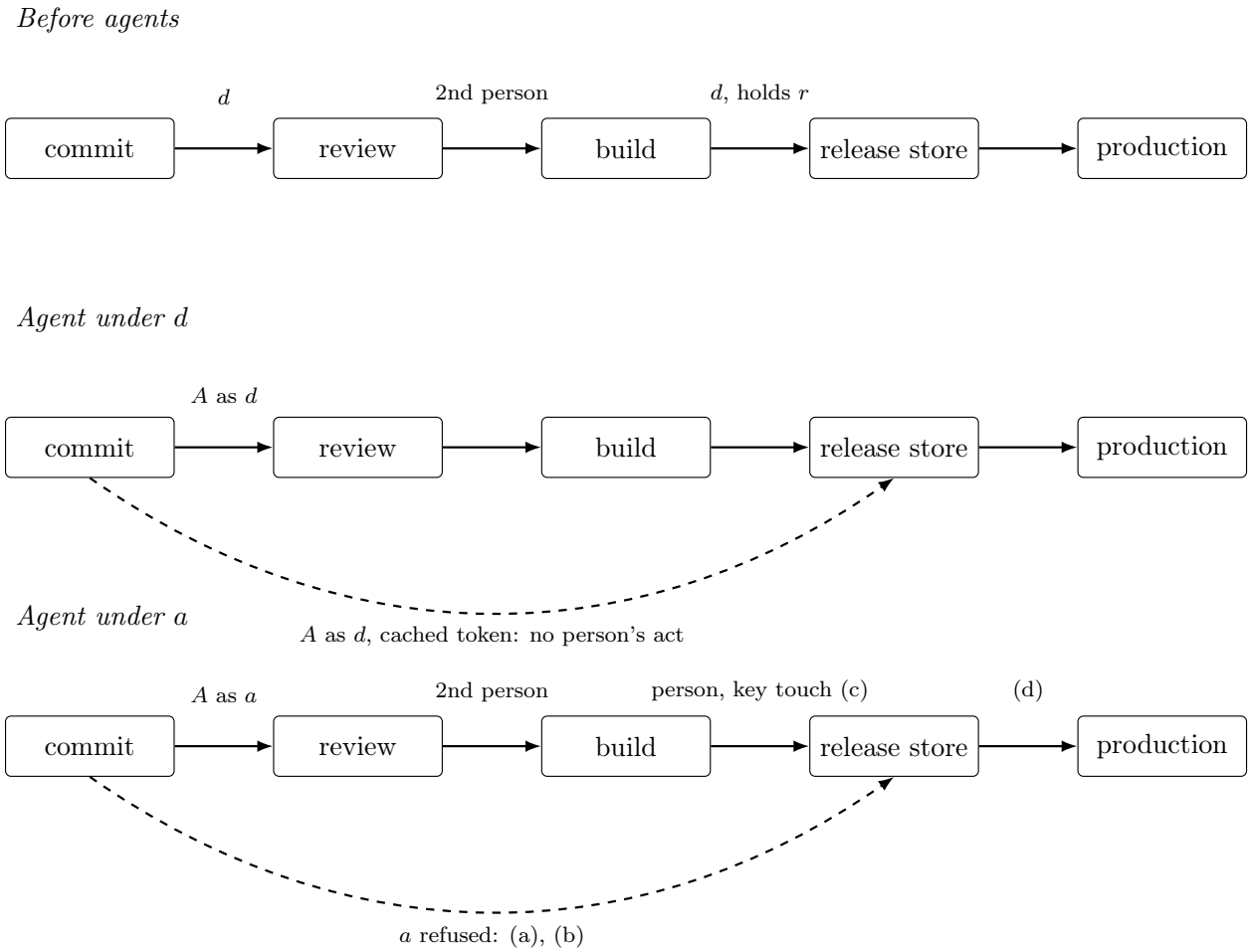


Figure 1: The chain from commit to production. Top: before agents, every arrow is a person's act. Middle: the agent runs as d, and a dashed path reaches the release store with no person's act. Bottom: under the rule the agent's identity a is refused at the store, and the release act needs a person.

Claim 3 (Every release is a person's act)**.** Under (c) and (d), every release act is a person's act, and so is every other change a release makes.

Argument. By (c) the release act requires an input only a person can supply, so by Definition 1 its issuer is that person, whatever identity carries the act. A pipeline that holds r issues the act only on that input, so the act is the person's who gave it. By (d) the other changes are part of the same act. □

Corollary 1. Under the rule, Invariant 1 holds for every chain to production that starts with A.

Argument. By Definition 2 the chain ends with a change in production. By (d) that change is made by the release act or is part of it. By Claim 3 that act is a person's, and its identity holds r because the permission system carried it out. □

Clauses (c) and (d) restore the invariant. Clauses (a) and (b) bound only the acts issued from a's

shell: code of a's that d pulls and runs, a changed build script or a test, carries d's identity, and d's cached token with it. So (c) carries the weight, and (a) and (b) add two things: a second boundary when the signature check is misconfigured, and the agent's own name in the log. Before the gate the agent does what a committer without r does: it commits, builds, tests, and deploys to the environments before the gate.

The recipe. On the machine, a is a separate user of the operating system, in no group that grants root, with no read access to d's credentials. The machine serves no credential to every user, as a cloud instance role does. a works in a clone of its own, and d runs nothing from it: d takes a's commits through the shared repository, as from any colleague, and builds them only after review or in a sandbox without d's credentials. No credential that holds r is in the repository's contents or history, and no token issued to a for the environments before the gate reaches past them. An agent inside an editor runs as the editor's user, so a runs in a terminal or a container of its own. In the cloud, a is assigned to the test account and to no other, and the denial of (b) is one of: a rule on the release store that refuses writes from every identity but the releaser's; an organisation-level rule on the production account; a protection on the branch the servers build from. Where d is the organisation's owner, (b) has no holder, and the team has the user boundary of (a) and the act of (c) only. For (c), the artifact is signed with a key on a hardware device that must be touched, and the deploy job the release starts checks the signature before it hands the artifact to production; or a second person approves, and the platform counts that approval against a list of persons, with a outside it. A pipeline that holds r acts only from the protected branch or environment, never from a branch or a tag that a can write. For (d), the database change and the restart of the workers [19] move out of the workstation script into the deploy job the release starts.

One team, twice. Five developers, one releaser, artifact promotion by a command on the releaser's laptop with a cached cloud token. The releaser's agent reads a library's README with a planted line that says to upload the current build to the release store. Before the rule, the agent composes the upload, the token is on the disk, the allow list covers the storage command, and the artifact is live; the log names the releaser. After the rule, the same upload is issued under a, the store refuses it, and the log names a; the real release waits for the key touch.

The rule assumes nothing about why the agent issues a command. It holds for a planted instruction, for an error, and for a model that pursues a goal of its own, since by A1 the permission system decides on the identity and the signature on the person, and neither asks for the reason. It holds within the two boundaries named at the end of this section. A model that attacks the boundary itself is the case of the open letter, and outside this paper.

The weakest link of the chain is then review, as it was before agents. Where review is absent, the rule still gives a person's act, a signature, a log entry under the right name, and a pause before production in which a person can look.

Why a condition. The rule is needed on a machine when two things hold: the developer holds r, and a program can supply everything the release act asks for. Where the developer cannot release, $\mathrm{perm}(A) = \mathrm{perm}(d)$ excludes r already. Where the release act already needs a person, the invariant holds, and (a) and (b) add the agent's name in the log and a second boundary. The rule costs the team one permission set and one deny, and each releaser's machine one identity and one operating-system user: about a day for the team and its first machine, by the author's estimate, and less for each next one. In a team where one person releases, that is one machine. Every other developer's agent is bounded by that developer's rights, and a second identity there adds nothing for the release act. The environments before the gate are then inside every agent's reach, so they must hold no production data and no network path to production.

Why personal. A shared agent identity outlives the person who leaves, and its log entries name no person. One identity per releaser who runs an agent keeps both properties of personal credentials: removal with the person, and a name in the log.

Why the account and not the harness. A harness rule chooses which command runs, and the developer widens it at the agent's pace. A sandbox deny is a list of paths and hosts, and the credential also lives in environment variables, sockets and keyrings; the user boundary needs no list, and it names a in the log. Hardy's remedy for the confused deputy is capabilities: the deputy holds no authority of its own and acts only with what the requester hands it [3]. CaMeL takes that step inside the agent. The rule takes the smaller step of least privilege outside it: the deputy keeps an authority of its own, cut to what its task needs, and the resource decides on that authority. Automation that runs off the developer's machine already works this way: a GitHub App "can act independently of a user" with "narrow, specific permissions" [13], and agents hosted in a vendor's cloud act as identities of their own. That design stops at the developer's machine. This paper extends it there.

Why the release model matters. The rule needs a release act that is distinct from the commit. Where software is built once and the artifact is promoted to production, the promotion is that act and the release store holds the deny of (b) [18, 19]. Where every server builds from a branch, the merge into that branch is the release act, r is the merge permission, and (b) is the branch protection. The test of Table 1 applies to that protection. A required status check asks for a green build, and a program supplies one. A required approval by a second person asks for that person, and a program does not. The agent's identity in source control must then be its own, since the developer's token merges as the developer. The protection must be an organisation rule that the repository's administrators cannot bypass. It must require a second person's approval of the last push, counted against a list of persons that excludes a, and no workflow may approve. The person's act is then the approval, and anyone may press merge. Where neither act exists, every commit is a release and the rule cannot be applied.

What the rule leaves open. The rule rests on two boundaries, the operating system's user boundary on the developer's machine and the cloud's permission system, and a model that defeats either is outside it. The agent can ask the developer to run a command or to touch the key, and it writes the description a reviewer reads. A release made on a person's misled judgement is a person's act, and the rule leaves it to review. Source code leaving through the agent's network access, and secrets readable on the developer's machine, are separate matters.

5 Relation to Published Rules

OWASP's Excessive Agency entry asks for the minimum permissions, complete mediation in the systems the agent reaches, and a person's approval of high-impact actions [20]. Willison's lethal trifecta names the three capabilities that together make an injected agent dangerous: access to private data, exposure to untrusted content, and a way to communicate out [21]. Meta's Rule of Two turns that into a design rule: in one session an agent may have at most two of untrusted input, access to sensitive systems or private data, and the power to change state or communicate out [22]. A coding agent has all three by its job: it reads text from outside, it holds the source and the test credentials, and it changes state. Meta's text for that case is a person's approval of the consequential act, and clause (c) is that approval, placed at the release. Clauses (a) and (b) bound what the other two legs reach: under a the sensitive system the agent can change is the

test environment, and production is out of its reach. The resource-decides principle of Section 4 is OWASP's complete mediation. This paper adds the construction for an agent in the developer's own shell, and the condition that names the machines that need it.

6 Conclusion and Recommendation

The rule applies to the machines of releasers whose release act a program can pass, and to no others. Table 2 gives each clause with the threat it closes and the check that shows it holds.

Table 2: The four clauses, what each closes, where it is enforced, and how to check it.

Clause	Closes	Enforced by	Check
(a) own identity	the agent inheriting the releaser's credentials	the operating system's users; the cloud's identities	from a's shell, a release to a test target is refused and the log names a
(b) deny above d	the releaser widening the deny at the agent's pace	a store rule, an organisation rule, or a branch rule held above d	d cannot edit the rule; the policy simulator refuses r for a
(c) person's input	the three program-suppliable checks of Table 1, and pipelines that hold r	the signature check in the deploy job, or an approval counted against persons	a release without the touch or the approval is refused
(d) one act	side stores, database changes and restarts outside the gate	the deploy job the release starts	no production store accepts a's writes; no release step runs by hand

In order: list who holds r and which of them run an agent on the machine where they sign in; on each such machine, put (a) and (b) in place; make the release act a person's act and route every release change through it; keep review, since the rule returns the chain to the strength of its review and adds nothing to it.

Until the rule is in place, two measures take minutes and change nothing for the team. An ask rule in the harness on the production release command makes it prompt again. A sign-out after each production release clears the sign-in token, so that it does not outlive the act. Both shorten the window without closing it.

Author Note

The author has worked in software development and architecture since the 1990s, twelve of those years in regulated environments, and is the release engineer of the system in the case study [19], where he runs a coding agent on the release machine. Drafts were produced by Claude from the author's notes; the author edited every sentence and is responsible for every claim. October 2026.

References

[1] Saltzer, J. H., & Schroeder, M. D. (1975). The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9), 1278–1308. https://doi.org/10.1109/PROC.1975.9939

[2] Clark, D. D., & Wilson, D. R. (1987). A Comparison of Commercial and Military Computer Security Policies. In *Proceedings of the 1987 IEEE Symposium on Security and Privacy*, 184–194. https://doi.org/10.1109/SP.1987.10001

[3] Hardy, N. (1988). The Confused Deputy (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4), 36–38. https://doi.org/10.1145/54289.871709

[4] Akhawe, D., & Felt, A. P. (2013). Alice in Warningland: A Large-Scale Field Study of Browser Security Warning Effectiveness. In *Proceedings of the 22nd USENIX Security Symposium*. https://www.usenix.org/conference/usenixsecurity13/technical-sessions/presentation/akhawe

[5] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*. arXiv:2302.12173. https://doi.org/10.48550/arXiv.2302.12173

[6] Zhan, Q., Liang, Z., Ying, Z., & Kang, D. (2024). InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. https://aclanthology.org/2024.findings-acl.624

[7] Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., & Tramèr, F. (2024). AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*. arXiv:2406.13352. https://doi.org/10.48550/arXiv.2406.13352

[8] Debenedetti, E., Shumailov, I., Fan, T., Hayes, J., Carlini, N., Fabian, D., Kern, C., Shi, C., Terzis, A., & Tramèr, F. (2025). Defeating Prompt Injections by Design. arXiv:2503.18813. https://doi.org/10.48550/arXiv.2503.18813

[9] Shi, T., He, J., Wang, Z., Li, H., Wu, L., Guo, W., & Song, D. (2025). Progent: Securing AI Agents with Privilege Control. arXiv:2504.11703. https://doi.org/10.48550/arXiv.2504.11703

[10] National Vulnerability Database. (2025). *CVE-2025-54135: Cursor allows writing in-workspace files with no user approval in versions below 1.3.9; an attacker can chain an indirect prompt injection to write the settings file and trigger remote code execution.* Published 5 August 2025, CVSS 9.8. https://nvd.nist.gov/vuln/detail/CVE-2025-54135

[11] Stack Overflow. (2026). *2026 Developer Survey: AI.* Results published 6 October 2026, read 15 October 2026. https://survey.stackoverflow.co/2026/ai/data

[12] Anthropic. (2026). *Claude Code: Permissions.* Read 14 October 2026. https://code.claude.com/docs/en/permissions

[13] GitHub. (2026). *About creating GitHub Apps.* GitHub Docs, read 15 October 2026. https://docs.github.com/en/apps/creating-github-apps/about-creating-github-apps/about-creating-github-apps

[14] Google Cloud. (2026). *Binary Authorization for Borg: How Google Verifies Code Provenance and Implements Code Identity.* Page updated September 2026, read 15 October 2026. https://docs.cloud.google.com/docs/security/binary-authorization-for-borg

[15] Scientific community. (2026). *An Open Letter on AI and Cybersecurity from the Scientific Community.* October 2026; 483 signatories as read on 15 October 2026. https://ai-security-letter.org/

[16] Okafor, C., Schorlemmer, T. R., Torres-Arias, S., & Davis, J. C. (2022). SoK: Analysis of Software Supply Chain Security by Establishing Secure Design Properties. In *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses (SCORED).* https://doi.org/10.1145/3560835.3564556

[17] Open Source Security Foundation (OpenSSF). (2023). *Supply-chain Levels for Software Artifacts (SLSA), Version 1.0.* Read 15 October 2026. https://slsa.dev/spec/v1.0/

[18] Gavrilov, V. (2026). *Artifact Promotion as a Control Model for Stable Cloud Deployment: Build on Target Machines vs. Build Once and Promote Artifacts.* Zenodo. https://doi.org/10.5281/zenodo.20451077

[19] Gavrilov, V., & Ylli, E. (2026). *The Artifact Promotion Control Model: An Implementation Case Study.* arXiv:2609.35891. https://arxiv.org/abs/2609.35891

[20] OWASP Gen AI Security Project. (2025). LLM06:2025 Excessive Agency. In *OWASP Top 10 for LLM Applications 2025.* Read 15 October 2026. https://genai.owasp.org/llmrisk/llm062025-excessive-agency/

[21] Willison, S. (2025). *The Lethal Trifecta for AI Agents: Private Data, Untrusted Content, and External Communication.* 16 June 2025. https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/

[22] Meta AI. (2025). *Agents Rule of Two: A Practical Approach to AI Agent Security.* 31 October 2025. https://ai.meta.com/blog/practical-ai-agent-security/